ACME: Agentic Contract-Based Multilevel Engineering for High-Assurance Complex Systems

PI: Pierluigi Nuzzo, Associate Professor, Department of Electrical Engineering and Computer Sciences, University of California, Berkeley

Cash funding needed: $80,000–$100,000
AWS Promotional Credits needed: $25,000–$40,000

Amazon contact (optional): [Amazon contact name], [email]

Abstract - Large language model (LLM) agents are increasingly employed to design and verify complex systems, such as register-transfer level (RTL) hardware, controllers for teams of robots, and workflows of cooperating software agents. Their design requires decomposing system-level requirements into component-level specifications, which is a hard problem for temporal specifications: distributed reactive synthesis is undecidable in general, and finding a decomposition whose components are realizable and whose composition is correct is still largely an art. In this project, we propose a framework, termed Agentic Contract-Based Multilevel Engineering (ACME), for sound decomposition of hardware, robotic, and agentic software systems based on assume-guarantee (A/G) contracts. Agents propose contracts at every level of a design hierarchy, and a contract engine with model checking and synthesis back ends decides whether each step is sound. ACME follows a three-pronged approach: a theory of decomposition for temporal contracts, including sound approximations of the contract quotient and circular reasoning across components; methods to decompose agentic tasks into roles with contracts that provably refine the task, optimized jointly with the cost of executing the roles; and formal feedback that localizes failures, repairs decompositions, and reuses certified components across designs. ACME will be built on CHASE, our open-source contract-based design framework, and evaluated on hardware, multi-robot, and multi-agent software benchmarks.

Keywords – assume-guarantee contracts, compositional verification, reactive synthesis, GR(1), neurosymbolic reasoning, LLM agents, hardware formal verification, autoformalization.

Introduction

Next-generation computing systems, from custom silicon to fleets of autonomous robots and workflows of cooperating AI agents, are increasingly designed and verified with the help of LLM agents. LLMs generate assertions and invariants for hardware [1, 2] and programs [3], while the decomposition of the problem, where most of the difficulty lies, rests on the agent. End-to-end properties of large systems, such as in-order data delivery across a chain of hardware modules or mutual exclusion over a fleet of robots, often remain undetermined when checked on the flat design. In our experience, agents that decompose such properties without supervision may report timed-out runs as proofs or rely on circular assumptions across lemmas. Agentic software faces the same problem in another form: when an agent delegates subtasks or compacts its context, the rules it must follow are restated in natural language and may be lost [4].

Contract-based design offers a principled foundation for decomposition in all these domains. An A/G contract specifies what a component guarantees under assumptions on its environment [5]. A decomposition is correct when the composition of the component contracts refines the system contract, and the algebra of contracts provides operators, such as the quotient, that compute the weakest specification a missing component must satisfy [6, 7]. Existing contract-based tools, including CHASE [8], Pacti [9], and OCRA [10], assume that contracts are written by human designers, and existing decomposition methods for synthesis rely on a fixed library of components [11]. Letting agents write the contracts raises several scientific questions. The existence of a decomposition into realizable components is undecidable for general architectures [12], so an agent may search indefinitely unless each failure is explained in a form that excludes whole families of candidates. The contract operators must also remain computable within the fragments

that synthesis tools accept; for instance, the guarantee of the quotient of two GR(1) contracts contains the negation of assumptions and in general falls outside the GR(1) fragment.

In this project, we propose a framework, termed ACME, aiming to address these challenges by following a three-pronged approach that builds on our prior work. CHASE [8] provides an extensible contract algebra and interfaces to verification and synthesis tools, and has recently been extended with an interface for LLM agents. Our work on contract-based architecture exploration [13, 14] has shown that infeasibility certificates can be generalized via subgraph isomorphism to prune large design spaces, and received the Best Paper Award at DATE 2024. ACIES [15] uses LLM agents to decompose multi-robot requirements into GR(1) contracts that are certified and synthesized into controllers, and our ongoing work applies the same principle to RTL verification. If successful, this project will lay the foundations for agentic design flows in which every decomposition proposed by an AI agent, for a hardware property, a robot mission, or a software task, comes with a certificate that a solver can check, and in which certified components are reused across designs.

Methods

ACME represents a design as a directed acyclic graph of contracts, where each node is a component with a contract over its ports. At an internal node, an agent proposes one contract for each subcomponent. The proposal is accepted only if the component contracts are consistent, bounded checks on the actual components find no counterexample, and their composition refines the parent contract. Each subcomponent is then discharged by a back end or, if this does not terminate within a budget, further decomposed. When a node cannot satisfy its contract, it returns a witness to its parent, which revises the decomposition while certified subtrees are preserved. The leaves depend on the domain: a hardware module that a model checker proves directly, a task that one LLM agent completes or a piece of deterministic code, or a robot controller certified by, e.g., a control barrier function or Hamilton-Jacobi reachability.

Thrust 1: Decomposition of Temporal Contracts. GR(1) synthesis [16] is the common back end of our robotic and agentic flows.

RQ 1. *Given a system contract in the GR(1) fragment and an architecture, devise (a) conditions under which a decomposition into realizable component contracts exists, (b) sound approximations of the contract quotient within GR(1), and (c) methods to reason about components that depend on each other circularly.*

Task 1. Decidable classes and quotient approximations. We will identify classes of architectures and contracts for which the existence of a decomposition is decidable, starting from pipeline architectures, for which distributed synthesis is decidable [12], and use the agent as a search heuristic outside these classes. We will develop GR(1) approximations that strengthen the guarantees of the exact quotient: they can be solved directly by a synthesis tool, and any controller synthesized for them is correct for the composition.
Task 2. Circular reasoning and learned constraints. We will restrict the guarantees used as assumptions to safety properties, adopt a circular A/G rule with an inductive side condition [17, 18], and characterize the decompositions it admits. As learned clauses prune the search of a SAT solver, we will generalize the unrealizable core of a failed decomposition, via subgraph isomorphism [13], into a constraint that excludes every candidate assigning an isomorphic component the same terms.

Thrust 2: Decomposition of Agentic Tasks. Long-horizon tasks with hundreds of tools require delegation and context compaction, which can lose the rules an agent must follow [4], and whether one agent or several perform better depends on the task [19].

RQ 2. *Given a task contract over the actions of an agentic workflow, devise methods to (a) decide whether and how the task is delegated to roles, (b) generate role contracts whose composition refines the task contract, and (c) optimize the decomposition jointly with the cost of executing the roles.*

Task 3. Contract-based delegation. A parent agent passes to each role a task, a context, and contracts in linear temporal logic on finite traces (LTL_f) [20], inherited or generated, and the engine checks that their composition refines the parent contract. A role may need weaker assumptions, stronger guarantees, and conditions on its siblings, such as sending a confirmation only after a payment; the quotient of the parent contract by the contracts of the siblings gives the weakest admissible contract. A compacted context is accepted only if its contracts refine those of the full context. Vacuity [21] and well-separation [22] checks reject faulty formalizations, and the choice between one agent and several is cast as contract-based architecture exploration [13, 14]. *Task 4. Joint optimization with inference cost.* The states reachable in a role contract determine which tools the role may call next. We will use them to partition the key-value cache of the model into modules aligned with contract states [23, 24] and to prefetch modules for reachable states, so that dynamic tool lists do not invalidate cached prefixes, and include this cost in Task 3. Enforcement of the contracts at run time reduces to monitoring [25] and is outside the scope of this project.

Thrust 3: Formal Feedback, Repair, and Reuse.

RQ 3. *Given a failed decomposition step, devise methods to (a) return feedback that localizes the failure, (b) repair the decomposition with as few agent calls as possible, and (c) reuse certified components across designs.*

Task 5. Layered feedback. The engine separates a formalism-independent contract algebra, checks reduced to validity or satisfiability, and formalism-specific analyses such as unrealizable cores; every failed check returns a counterexample or a minimal core. A Model Context Protocol (MCP) server and an agent skill expose all operations to agents, no LLM is part of the engine, and bounded runs count as undetermined. *Task 6. Repair, reuse, and coverage.* Refinement failures are first repaired with the quotient, and the agent is invoked only when this repair fails. Certified components are reused after their assumptions are rechecked, and contract coverage is measured by fault injection.

Evaluation and preliminary results. In hardware, component contracts are SystemVerilog assertions (SVAs) with ghost-state models, verified with Cadence Jasper and the open-source SymbiYosys. Our prototype certified an end-to-end in-order delivery property of a HierSVA asynchronous link with 25 Jasper runs and about 16,000 LLM input tokens, which flat Jasper did not decide in the same time; the decomposition found at FIFO depth 8 certified depths 16 to 512 with no further LLM calls, and an agent without the protocol used 24 times as many tokens without completing the proof. In multi-robot control, ACIES [15] decomposes mission requirements into GR(1) contracts and synthesizes controllers; in a three-level decomposition with 42 nodes, the contract library closed 26 of the 39 leaves. For agentic software, we will add delegation, compaction, and cache-aware tool exposure one at a time to a single-agent executor on long-horizon tasks from AppWorld [26], against summarization-based compaction.

Expected results

Months	Research result	Deliverable
1–3	Quotient approximations (Task 1); layered feedback (Task 5)	Technical report; CHASE release with MCP server and agent skill
4–6	Circular A/G rule with a soundness proof; learned constraints (Task 2)	Open-source hardware flow with SymbiYosys on AWS; RTL benchmark with faulty variants
7–9	Contract-based delegation and compaction (Task 3); repair and reuse (Task 6)	Hardware and multi-robot evaluation; conference paper

Months	Research result	Deliverable
10–12	Contract-guided caching and joint optimization (Task 4); coverage (Task 6)	Agent evaluation; tool paper on CHASE; talk to Amazon teams

Beyond the first year, we expect these results to extend to further decidable classes of architectures, to contracts with probabilistic guarantees, and to design flows in which the three domains share certified components. All code, benchmarks, and run traces will be released under open-source licenses.

Funds needed

Cash funding (\$80,000–\$100,000). Partial support of one graduate student researcher working on the theory, the framework, and the benchmarks (stipend and fees, [amount]), and travel to present the results at conferences ([amount]). The upper end of the range would add partial support of a second student working on Thrust 2. No indirect expenses are included.

AWS Promotional Credits (\$25,000–\$40,000). Amazon Bedrock for agent inference, since our hardware and robotic pipelines already invoke Claude models through Bedrock; Amazon EC2 compute-optimized instances for parallel runs of nuXmv, slugs, and SymbiYosys; and Amazon S3 for benchmarks and run traces. Experiments that require Cadence Jasper will run on campus license servers.

Additional information

1. Application domain and analysis techniques. Compositional verification and design of hardware, multi-robot systems, and LLM agents, using A/G contracts, linear temporal logic (LTL) and LTL_f model checking (nuXmv), satisfiability modulo theories (SMT) reasoning, GR(1) synthesis (slugs), and RTL property verification (Jasper, SymbiYosys).

2. Current applications. CHASE has been used for requirement formalization and validation of cyber-physical systems [8], contract-based architecture exploration [13, 14], and consistency and availability analysis of distributed systems in collaboration with Lockheed Martin [27]. Our RTL flow runs on OpenTitan and HierSVA designs.

3. Potential applications to Amazon. Verification of AWS silicon; controllers with guarantees for Amazon Robotics fleets sharing chargers and aisles; multi-agent workflows on Amazon Bedrock whose obligations are preserved across delegation, at lower serving cost; and consistency and vacuity analysis of formalized policies, complementary to Automated Reasoning checks in Bedrock Guardrails.

4–5. Prior Amazon Research Awards. The PI has not received an Amazon Research Award, and the project team has not received cash funding or AWS Promotional Credits from Amazon in the past three years.

6. Assumptions and soundness. A certified result is sound when (i) the back ends return correct verdicts; (ii) the translation of contracts into each back end preserves their semantics, which we ensure by restricting SystemVerilog assertions to an LTL-expressible subset; (iii) every composition step is acyclic or satisfies the side condition of the circular rule in Thrust 1; and (iv) the top-level specification is correct, which is why extracted contracts are checked for consistency and vacuity and confirmed by the user. Bounded results are never counted as proofs, LLM outputs are never trusted, and quotient approximations are used only in the direction that preserves refinement.

7. Tools. *License:* CHASE and the CHASE MCP server are released under the BSD 3-Clause license, the RTL verification flow under Apache 2.0, and ACIES under the MIT license. *On-boarding material:* a website with a usage guide and API reference, a Docker image, and the agent skill; a tutorial will be added. *Maintenance:* over 90 commits with continuous integration in the last three months; one active contributor.

Appendix A – Reference

[1] Orenes-Vera, Marcelo, et al. “AutoSVA: Democratizing Formal Verification of RTL Module Interactions.” DAC. 2021.

[2] Kang, Minwoo, et al. “FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware.” arXiv:2410.23299. 2024.

[3] Wu, Haoze, Clark Barrett, and Nina Narodytska. “Lemur: Integrating Large Language Models in Automated Program Verification.” ICLR. 2024.

[4] Cemri, Mert, et al. “Why Do Multi-Agent LLM Systems Fail?” arXiv:2503.13657. 2025.

[5] Benveniste, Albert, et al. “Contracts for System Design.” Foundations and Trends in Electronic Design Automation. 2018.

[6] Incer, Inigo, Albert Benveniste, and Alberto Sangiovanni-Vincentelli. “Some Algebraic Aspects of Assume-Guarantee Reasoning.” arXiv:2309.08875. 2023.

[7] Incer, Inigo. “The Algebra of Contracts.” Ph.D. thesis, UC Berkeley, EECS-2022-99. 2022.

[8] Nuzzo, Pierluigi, et al. “CHASE: Contract-Based Requirement Engineering for Cyber-Physical System Design.” DATE. 2018.

[9] Incer, Inigo, et al. “Pacti: Scaling Assume-Guarantee Reasoning for System Analysis and Design.” arXiv:2303.17751. 2023.

[10] Cimatti, Alessandro, Michele Dorigatti, and Stefano Tonetta. “OCRA: A Tool for Checking the Refinement of Temporal Contracts.” ASE. 2013.

[11] Iannopollo, Antonio, Stavros Tripakis, and Alberto Sangiovanni-Vincentelli. “Specification Decomposition for Synthesis from Libraries of LTL Assume/Guarantee Contracts.” DATE. 2018.

[12] Pnueli, Amir, and Roni Rosner. “Distributed Reactive Systems Are Hard to Synthesize.” FOCS. 1990.

[13] Xiao, Yifeng, et al. “Efficient Exploration of Cyber-Physical System Architectures Using Contracts and Subgraph Isomorphism.” DATE. 2024.

[14] Xiao, Yifeng, and Pierluigi Nuzzo. “Contract-Based Architecture Exploration of Cyber-Physical Systems via Satisfiability Modulo Convex Programming.” DATE. 2026.

[15] Xiao, Yifeng, et al. “ACIES: A Self-Evolving Agentic Contract-Based Framework for Cyber-Physical System Control Synthesis.” REALM Workshop at EMNLP. 2026.

[16] Bloem, Roderick, et al. “Synthesis of Reactive(1) Designs.” Journal of Computer and System Sciences. 2012.

[17] McMillan, Kenneth L. “Circular Compositional Reasoning about Liveness.” CHARME. 1999.

[18] Namjoshi, Kedar S., and Richard J. Trefler. “On the Completeness of Compositional Reasoning.” CAV. 2000.

[19] Kim, Yubin, et al. “Towards a Science of Scaling Agent Systems.” arXiv:2512.08296. 2025.

[20] De Giacomo, Giuseppe, and Moshe Y. Vardi. “Linear Temporal Logic and Linear Dynamic Logic on Finite Traces.” IJCAI. 2013.

[21] Maoz, Shahar, and Rafi Shalom. “Inherent Vacuity for GR(1) Specifications.” ESEC/FSE. 2020.

[22] Maoz, Shahar, and Jan Oliver Ringert. “On Well-Separation of GR(1) Specifications.” FSE. 2016.

[23] Gim, In, et al. “Prompt Cache: Modular Attention Reuse for Low-Latency Inference.” MLSys. 2024.

[24] Yao, Jiayi, et al. “CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion.” EuroSys. 2025.

[25] Xiao, Yifeng, and Pierluigi Nuzzo. “Symbolic Temporal Supervision of LLM Agents Using Contracts.” arXiv:2609.18128. 2026.

[26] Trivedi, Harsh, et al. “AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents.” ACL. 2024.

[27] Xiao, Yifeng, et al. “Contract-Based Consistency and Availability Analysis for Distributed Cyber-Physical Systems.” MEMOCODE. 2026.